

Model Optimization for Infra Efficiency

Faster, Cheaper, Greener ML at Scale

Practical strategies for ML engineers to reduce latency, cost, and environmental impact while maintaining model quality.



Why Optimize?



Production constraints

Latency, throughput, memory, cost dictate what's viable in production

Resource-constrained environments

Edge & mobile need **tiny, low-power** models to function effectively

Infrastructure economics

Datcenters pay for **GPU hours & energy** - direct impact on budget

Optimization = same (or similar) quality at lower resource use

Optimization Toolkit

1

Quantization

Convert to lower precision weights/activations
(FP32→FP16→INT8→INT4)

2

Pruning

Remove redundant weights/structures to create sparser networks

3

Knowledge Distillation

Train smaller "student" models that learn from larger "teacher" models

1

Sparsity

Leverage zero-value structure patterns for computational speedups

2

Compilation

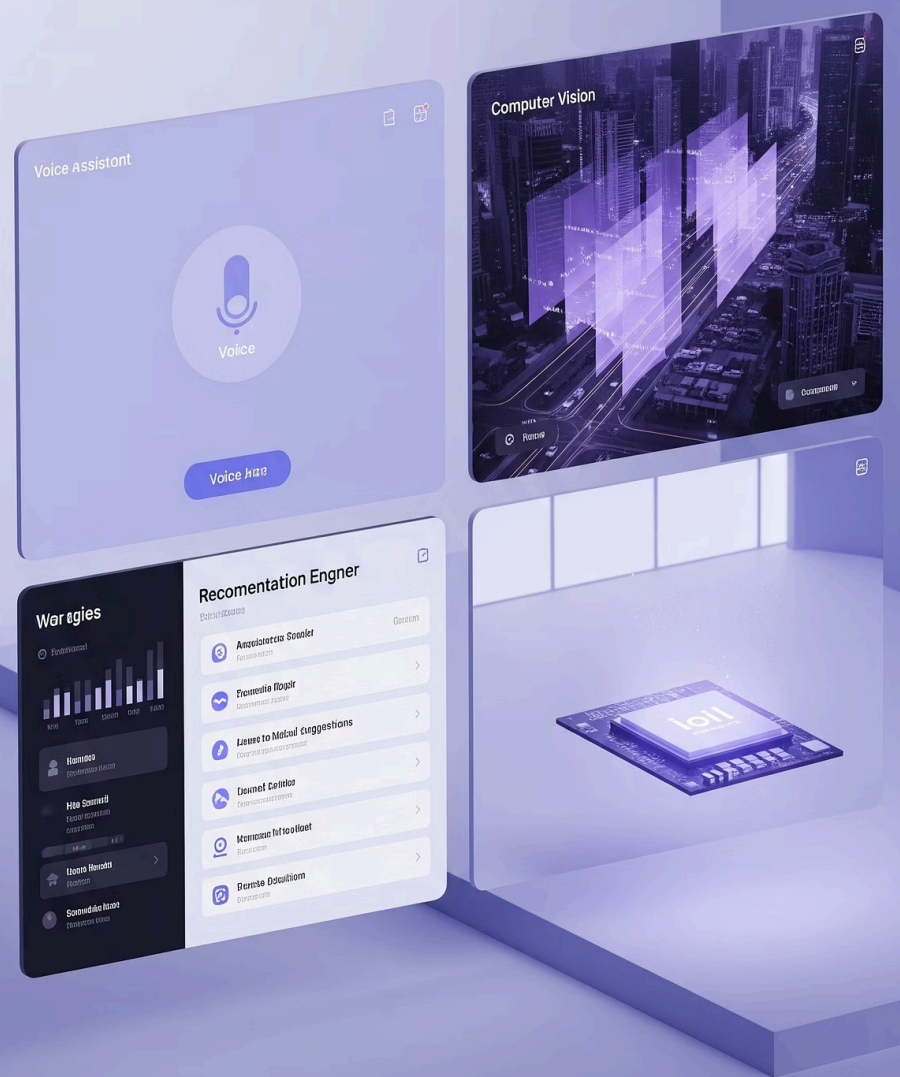
Graph-level fusion & kernel selection (TensorRT/ONNX Runtime/XLA)

3

Caching

KV-cache, embedding/prompt caches for LLMs to avoid redundant computation

Real Time AI Atpplications



Where It Pays Off



Inference Services

Optimize for p95/p99 latency and QPS (queries per second), especially for high-traffic endpoints



Real-time Applications

Voice assistants, computer vision systems, and recommendation engines that require sub-second responses



Batch Scoring

Reduce cost per million predictions for large-scale inference jobs running on cloud infrastructure



Edge/IoT Deployment

Extend battery life and operate within thermal limits of resource-constrained devices

Core Trade-offs

Understanding the optimization balance beam

Accuracy vs Latency/Size

How much quality degradation is acceptable for performance gains?

Online Quality vs Offline Metrics

Optimize for lab metrics or real-world A/B test performance?



Portability vs Hardware Gains

Generic optimizations or tailored to specific accelerators?

Development Speed vs Toolchain Complexity

Simple workflows or complex, multi-step optimization pipelines?

The **right balance** depends on your specific deployment constraints and business requirements

Workflow: Optimize Then Serve

Baseline



Measure latency, throughput, memory usage, and quality metrics of your unoptimized model

Select Method(s)



Choose optimization strategy: quantize / prune / distill / compile based on requirements

Calibrate & Fine-tune



Use representative data for quantization calibration; fine-tune to recover accuracy

Validate



Verify offline metrics & run edge-case tests to catch regressions

A/B Deploy

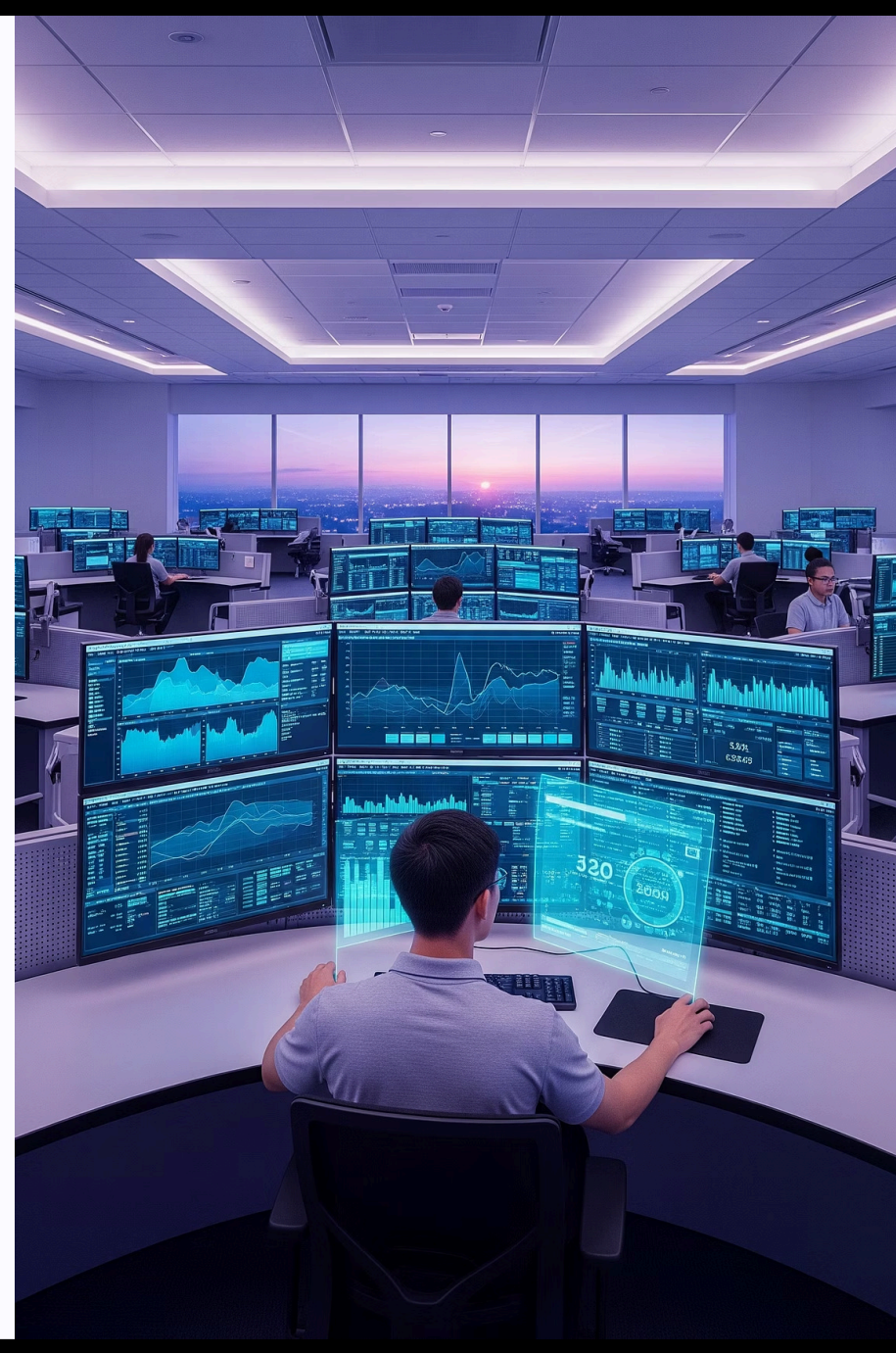


Roll out with guardrails and rollback plan in case of issues

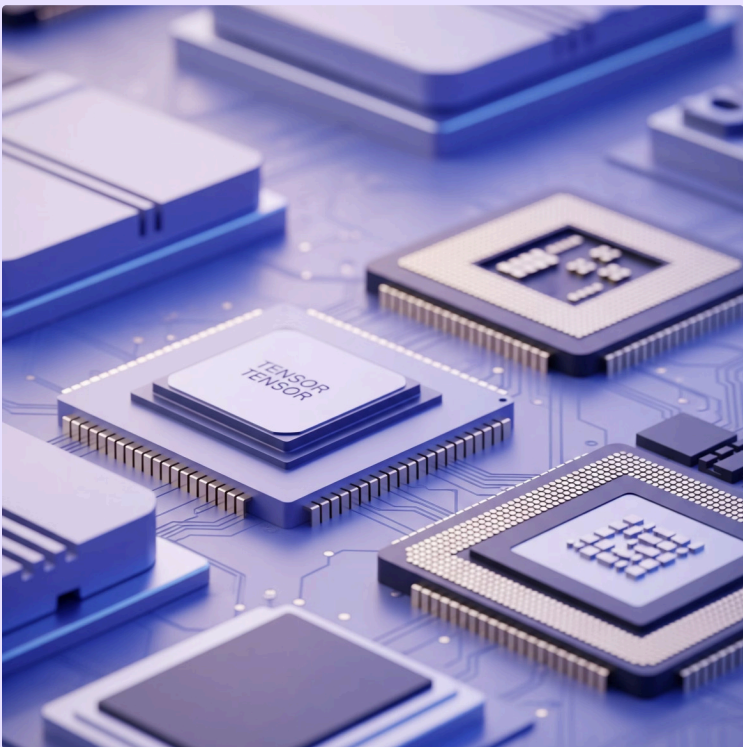
Monitor



Track drift, latency, cost, and error budgets in production



Hardware & Runtime Levers



Precision Engineering

- **Mixed precision** (BF16/FP16) on GPUs
- **INT8/INT4** on CPUs/NPUs/Edge (AVX512/VNNI, Tensor Cores)

Compiler Optimization

- TensorRT, ONNX Runtime, TVM, OpenVINO
- Kernel fusion, operator substitution, memory planning

Execution Strategy

- **Micro-batching** for higher throughput
- Concurrency controls and pinning models to memory

Pro tip: Different hardware accelerators have unique optimization pathways - know your target!



Data & Architecture Levers

Input Optimization

Shorter **sequence lengths** / reduced image resolutions where possible

Careful token pruning for LLMs can maintain quality with fewer tokens

Efficient Model Design

Use optimized backbones like MobileNet, EfficientNet, or DistilBERT

Consider efficient attention mechanisms (linear, sparse) for transformers

Parameter Efficiency

LoRA/PEFT: Parameter-efficient fine-tuning instead of full retrains

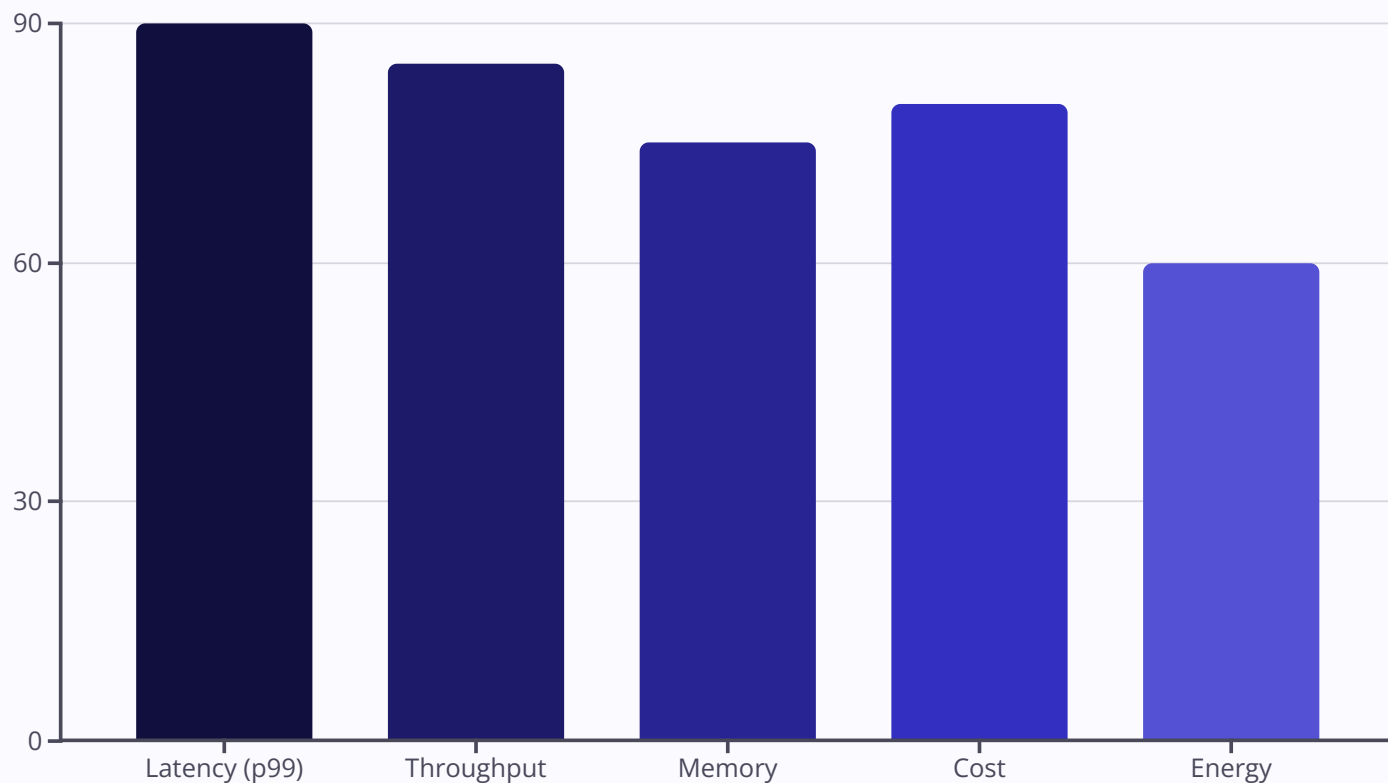
Adapter-based approaches that add minimal parameters to base models

Conditional Computation

MoE (Mixture of Experts): Activate experts sparsely for conditional compute

Early-exit architectures that finish inference when confidence is high

Measuring "Efficiency"



Performance Metrics

- **Latency:** p50/p95/p99 response times
- **Throughput:** requests/sec, tokens/sec

Resource Metrics

- **Memory:** peak GB, model size (MB)
- **Energy:** joules per inference

Business Metrics

- **Cost:** \$ per 1K inferences
- **Quality:** task metrics + human eval

Governance & Safety Considerations

⚠️ Post-optimization Validation

Re-validate safety guardrails and fairness metrics after optimization

Compressed models can exhibit different behaviors on edge cases

ℹ️ Artifact Management

Maintain **lineage & registry** for optimized artifacts

Track relationship between original and optimized models

✅ Contract Testing

Implement contract tests on inputs/outputs (schema, ranges)

Ensure API compatibility across model versions

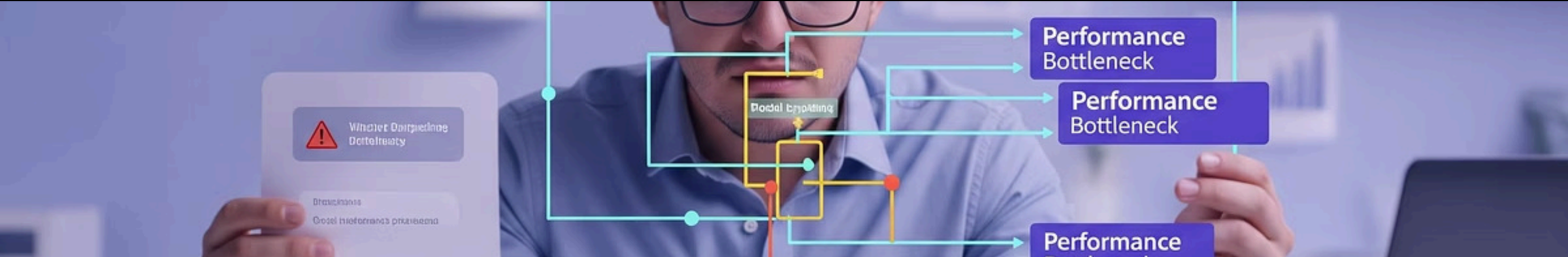
❌ Deployment Safety

Rollback plan + canary thresholds in continuous deployment

Progressive exposure of optimized models to catch issues early

Remember: Optimization should never compromise on safety or fairness requirements





Common Pitfalls

Quantization Without Calibration

Skipping proper calibration with representative data leads to severe accuracy drops, especially for INT8 and below

Ineffective Pruning Strategies

Unstructured pruning may look good in PyTorch but delivers no real speedup on target hardware without specialized support

Incomplete Profiling

Skipping **end-to-end profiling** (tokenize → inference → postprocess) misses bottlenecks that may dominate real-world latency

Focusing Only on Average Case

Ignoring tail latencies and multi-tenant interference can cause SLA violations in production despite good average metrics

Best Practices



Layered Approach

Start with **compiler/runtime** optimizations for "free" gains, then add quantization/pruning

Hardware-Aware Sparsity

Prefer **structured sparsity** patterns that your target hardware accelerators can exploit

Quality Recovery

Use **distillation** techniques to recover accuracy after aggressive compression

Realistic Testing

Validate with **production-like traffic patterns** before full rollout to catch load-dependent issues

Optimization stack: Layer techniques for multiplicative gains rather than applying them in isolation

Key Takeaways

Multi-layer Approach

Algorithm + runtime + hardware optimizations working together

Context Matters

Optimize for your specific **workload, device, and SLOs**



Balance Tradeoffs

Lower cost & latency with **minimal quality loss**

Continuous Process

Ongoing monitoring, governance, and refinement

Remember: The most efficient model is the one that meets your requirements with the least resources

Questions? Contact: mloptimization@company.com